LEANFM · A COMPACT ACCOUNT

LeanFM in Brief

Event streams, session types and CTL: a temporal-logic method for software engineering

Stream	Behavior is a stream of timestamped events, linked backward by *cause* and *prev*.
Protocol	Global session types, written in Lean, say which streams are legal. Recursion is the only control flow.
CTL	Formulas attached to protocols say what must, or may, happen next.
Picture	Every trace, witness and counterexample is drawn as an interaction diagram.

Rob Fielding · checked with LeanFM a7bc264 2026-11-02

Contents

1. The method on one page
2. Events: what the stream records
3. Protocols: session types in Lean
4. Interaction diagrams
5. CTL attached to protocols
6. Time
7. Checking: from protocols to verdicts
8. Witnesses and counterexamples
9. Finding a bug
0. Past and future
1. Working this way

Appendices: A. Notation · B. JSON forms · C. Reproducing this book

> **How to read the figures**
> Every diagram, verdict and formula reading in this book is **generated**. The examples are Lean modules written in LeanFM's session-type syntax. Lean elaborates and checks them, and they are compiled into task models and passed to LeanFM's own `check_ctl` and `check_trace` tools, the same code the LeanFM MCP server runs. Each figure carries a provenance line giving the tool and its verdict. Rebuilding the book reruns every check, and the build fails if a verdict stops matching the text.

1 The method on one page

Specify only what can be observed. Say which observations may follow which. Then ask, at each point, what the future must hold and what it can still hold.

A system is a set of **actors** exchanging **messages**. Every message that crosses an actor boundary is an **event**, and the run of a system is a **stream** of events. LeanFM specifies that stream and nothing behind it: no internal variables, no object graphs, no private state that an observer could not see.

1 · Events
Each event has an id, a timestamp, sender, receiver, message and fields. It points back to its cause and its prev.

2 · Protocols
A global session type over events, written in Lean. Sequence, choice, `include` and `rec`, with **recursion as the only loop**. `where` clauses relate fields.

3 · CTL
Each protocol carries formulas such as `AG EF terminal`: from here, finishing always stays possible.

4 · Diagrams
A trace is drawn as an interaction diagram: lifelines, arrows, cause brackets, protocol frames.

5 · Checking
Lean checks each protocol's choices and projections; the protocol compiles to a finite transition system on which CTL is decided, with a witness or counterexample path.

6 · Monitoring
Recorded streams are checked against the same protocols: links, timestamps and acceptance.

The two directions of time are handled differently, on purpose:

- **The past is data.** It is already in the stream. An event names the event that caused it and the event before it on its thread, so any fact about history is found by following links backward. There are no past-time operators.
- **The future is branching.** The protocol allows alternatives, so from any point there is a *tree* of legal continuations. CTL is the logic of that tree: `E` means some continuation, `A` means every continuation.

The result is a specification of manageable size that is easy to draw and check, and that can be handed to a person or an LLM as the complete,

durable statement of what a system must do. The running example throughout is a login session, written as three Lean protocols and the formulas attached to them:

```
protocol Session (Client, Server) {
  rec Login {
    l: Login from Client to Server;
    choice {
      Welcome from Server to Client cause l where valid l.user;
      include Work Client Server;
    } or {
      Denied from Server to Client cause l where !valid l.user;
      continue Login;
    }
  }
}

protocol Work (Client, Server) {
  rec Work {
    choice {
      include Request Client Server;
      continue Work;
    } or {
      o: Logout from Client to Server;
      Bye from Server to Client cause o;
    } or {
      x: Expired from Server to Client where x.idle ≥ 1800000;
    }
  }
}

protocol Request (Client, Server) {
  g: Get from Client to Server;
  choice {
    p: Page from Server to Client cause g
       where p.sid = g.sid ∧ p.time - g.time ≤ 2000;
  } or {
    n: NotFound from Server to Client cause g
       where n.time - g.time ≤ 2000;
  }
}

/-- What must, or may, happen: CTL formulas attached to protocols.
Each holds when an instance of its protocol begins, over every
continuation until that instance is finished. -/
def ctl : List (String × String) :=
  [ ("Session", "[ not Get AW Welcome ]")
  , ("Session", "[ not Get AU Welcome ]")
  , ("Session", "EF Bye")
  , ("Session", "AG EF terminal")
  , ("Session", "Get ==> AF (Page or NotFound)")
  , ("Work",    "AG EF terminal")
```

```
  , ("Work",    "AF terminal")
  , ("Request", "AF terminal") ]
```

This is an ordinary Lean module: `protocol` is syntax from LeanFM's `LeanFM.Session` library, and each protocol elaborates to a value of type `Global R` for any role type `R`. The rest of the book explains it one feature at a time.

2 Events: what the stream records

An event is one observation at one instant. Everything else, including state, duration, sessions and causality, is computed from events.

The event envelope is deliberately small:

`id`
: Unique within the stream.

`timeAt`
: Milliseconds on one observer's monotonic clock. Only differences matter.

`task`
: The conversation the event belongs to, such as one session. Many tasks interleave in one stream.

`src`, `dst`, `message`
: Who sent what to whom. Actors may be instances of a role: `Client(ann)` plays `Client`.

`prevIds`
: The immediately preceding event(s) on the same thread. A fork gives several events that share a prev; a join lists several prevs.

`causeIds`
: The event(s) this one answers or finishes. A response names its request; an end names its start.

```
{ "id": "b2", "prevIds": ["b1"], "causeIds": ["b1"], "task": "bob", "src": "Server",
"dst": "Client(bob)", "message": "Denied", "timeAt": 150 }
```

Two backward links, two jobs

prev is *order*. Following prev links reconstructs each conversation from an interleaved stream, without trusting arrival order. Two events are **concurrent** when neither can reach the other through the links.

cause is *bracketing*. A request and the response that names it enclose everything in between, like a call and its return. Brackets nest, so a flat stream becomes a tree of calls. Because the link names a specific event, the right request is matched even when many are in flight.

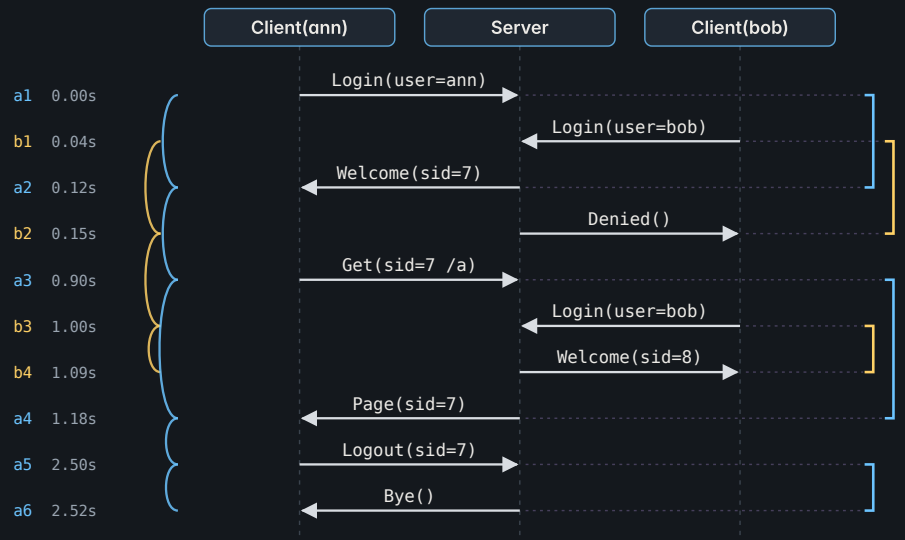


Figure 2.1 Two sessions interleaved in one stream. The left threads are prev links: blue for ann's task, gold for bob's. The right brackets are cause links, each joining a reply to its request. Bob's second login is bracketed inside ann's `get … page`: nested in time, but in a different task.

> LeanFM `check_trace`: **admissible** — 10 events over 2 task(s); 24 concurrent pair(s) · task ann: accepted=true, reached terminal=true; task bob: accepted=true, reached terminal=false

From the same stream, `check_trace` also returns the happens-before graph. Solid arrows are prev; dashed arrows are cause. Ann's and bob's events are unordered with respect to each other: LeanFM counts **24** concurrent pairs (for example a1‖b1, a1‖b2, a1‖b3, a1‖b4).



Figure 2.2 The causal graph exactly as `check_trace` emits it, rendered from its Mermaid output.

Timestamps

A timestamp is meaningful mainly in a *difference along a link*. The latency of a request is `reply.timeAt − request.timeAt`, with the reply found through its cause link. An idle gap is `e.timeAt − e.prev.timeAt`. The one rule the stream itself must obey is that no prev or cause link runs backward in time. Global order across tasks is not needed and is not claimed.

3 Protocols: session types in Lean

A protocol is a language whose letters are events. A global session type says which words are legal, and Lean checks that every role can play its part.

The messages are Lean structures, and their fields are what `where` clauses read. A misspelt message is an unknown identifier, not a new message:

```
-- The vocabulary: every message is a Lean structure, and its fields are
what guards read.
structure Login    where user : String  deriving TypeName
structure Welcome  where sid : Nat      deriving TypeName
structure Denied   where                deriving TypeName
structure Get      where
  sid : Nat
  path : String
  time : Nat
  deriving TypeName
structure Page     where
  sid : Nat
  time : Nat
  deriving TypeName
structure NotFound where
  sid : Nat
  time : Nat
  deriving TypeName
structure Logout   where sid : Nat      deriving TypeName
structure Bye      where                deriving TypeName
structure Expired  where idle : Nat     deriving TypeName

/-- Whether a user may log in: decided by the server, opaque to the
protocol. -/
def valid (user : String) : Bool := user != ""
```

A protocol body is built from a handful of statements:

`l: Login from Client to Server;`	an event; the optional name `l` lets later events refer to it
one statement after another	sequence
`choice { … } or { … }`	exactly one branch happens; the role that starts every branch decides which
`include Work Client Server;`	run another protocol here, with these roles; each protocol is written as if it were the whole thing
`rec Login { … continue Login; }`	a named block that `continue` repeats
`or { }`	an empty branch: nothing happens

An event may add `cause l`, meaning "this event answers `l`". That emits the cause link and opens or closes a bracket. It may also add `where e`, a Lean `Bool` over the fields of named events, such as `p.sid = g.sid`. It is type-checked when the protocol is written and evaluated over decoded payloads at check time (Chapter 6).

One role decides each choice

A choice does not say who decides: Lean reads it off the branches. The decider is the role that starts every branch, by sending its first message or by an event of its own, so there is only one way to write a choice and no name to get wrong. In `Login` the Server decides, because both branches start with its answer, `Welcome` or `Denied`. `Work` breaks that rule on purpose: the Server's `Expired` races the Client's next `Get` or `Logout`. No single role decides, and the session checker says so, naming who starts what:

```
no single role decides a choice, so it is a race: Client starts Get or Logout; Server
starts Expired
```

Here the idle-time bound is what settles the race in practice, and the book keeps the race to show the check. The usual fix makes the timeout physics, not a branch: a deadline on the channel, which the protocol handles (the full LeanFM book does this for TFTP).

Each role's part: the local type

Projection keeps, for one role, the events it sends or receives. Lean computes it for every role. Where the Server starts every branch of a choice, the Server *selects* and the Client is *offered* what arrives. For `Request`, the Server's part is:

```
receive Get from Client;
select {
  send Page to Client;
} or {
  send NotFound to Client;
}
```

and the Client's:

```
send Get to Server;
offer {
  receive Page from Server;
} or {
  receive NotFound from Server;
}
```

A local type is a production of its own: it is exactly the thread an actor runs for this session. When projection fails, as for the Server in `Work`, the role has no program that follows the protocol, and that is the error to fix.

There is no loop construct

There is no `while`, no Kleene star and no state variable. Repetition is a named block, `rec`, that `continue` repeats, in three shapes:

Continue at the end is a loop.
`Work` keeps serving requests: after a `Request`, `continue Work` starts the block again, so nothing accumulates.

Choice plus `continue` is a loop with an exit.
`Login` retries after `Denied` with `continue Login` and leaves through `Welcome`. The exit condition is simply the other branch.

`include` composes.
An included protocol runs in place, and whatever follows the `include` runs where it finishes. Between a request and its reply, as with `Upstream` below, it opens a bracket inside a bracket.

```
protocol Upstream (Proxy, Server) {
  choice {
    u: Fetch from Proxy to Server;
    Content from Server to Proxy cause u;
  } or {
  }
}

protocol FetchPage (Client, Proxy, Server) {
  g: Get from Client to Proxy;
  include Upstream Proxy Server;
  Page from Proxy to Client cause g;
}

def ctl : List (String × String) :=
  [ ("FetchPage", "AF terminal")
  , ("FetchPage", "EF Content") ]
```

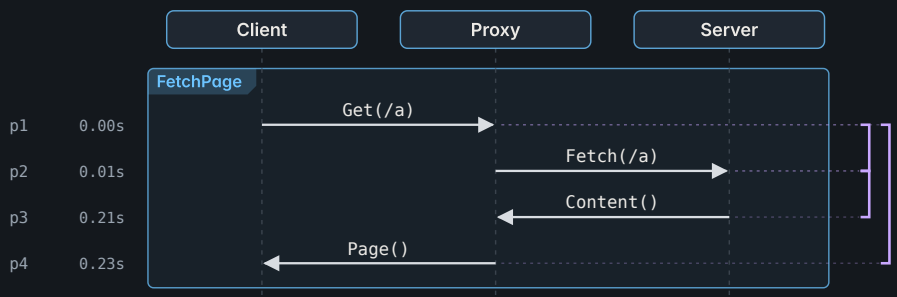


Figure 3.1 `Upstream`, included between `Get` and `Page`, nests its cause bracket (`Fetch` … `Content`) inside `FetchPage`'s (`Get` … `Page`). `Fetch` also names `Get` as its cause, which gives the third bracket.

> LeanFM `check_trace` : **admissible** — 4 events over 1 task(s); 0 concurrent pair(s)

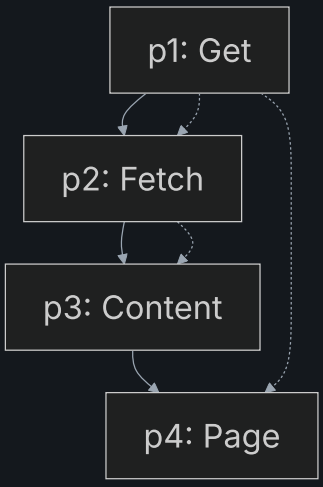


Figure 3.2 The same stream's causal graph from `check_trace`.

Why recursion? It makes loops *grammatical*, not operational. Every iteration of a `rec` block has a beginning and an end, which a diagram can frame and a formula can talk about. And the syntax keeps the transition system finite: `continue` ends its branch (nothing after it runs), so every repetition is in tail position, and unbounded nesting cannot be written at all.

4 Interaction diagrams

The one picture that matters. A trace, a witness and a counterexample are all streams, so all of them are interaction diagrams.

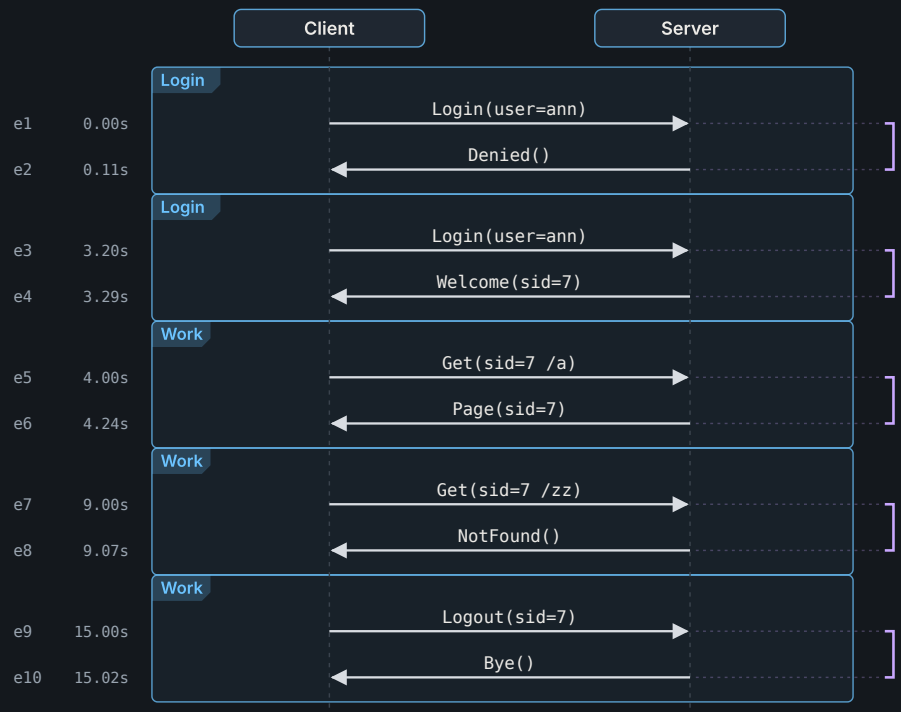


Figure 4.1 One session, parsed by the protocols of Chapter 1: a denied login, a retry, two requests and a goodbye.

> LeanFM `check_trace` : **admissible** — 10 events over 1 task(s); 0 concurrent pair(s) · task Session accepted, reached terminal=true

How to read it:

- **Lifelines** are actors. **Rows** are events in stream order. The left column shows each event's id and `timeAt`.
- An **arrow** is one message from sender to receiver, labeled with the message and its fields.
- A **bracket** on the right joins an event to the event it names as its cause. Nested brackets are nested calls.
- A **frame** is one protocol instance, or one iteration of a `rec` block, in the parse. Each `continue` starts the next frame at the same level (`Login`,

`Login`; `Work`, `Work`, `Work`), because repetition is a loop. An included protocol runs in place, so `Request`'s events sit inside the `Work` frame that included them.

- Red rows are violations. A dashed amber frame marks the part of a counterexample that **repeats forever**.

Frames are computed, not drawn: the trace is matched against the protocol, and every instance and iteration in the successful parse becomes a frame. A trace that does not parse has no frames, and its first unparseable event is the error to report.

5 CTL attached to protocols

A protocol generates a tree of possible futures. CTL quantifies over that tree. Attaching a formula to a protocol says where the question is asked.

Every temporal operator in CTL pairs a **path quantifier** with a **temporal word**:

Formula	Reading	Typical use
`EF p`	possibly, eventually p	reachability, with a witness
`AF p`	on every future, eventually p	termination, response
`AG p`	on every future, at every point, p	safety invariant
`EG p`	on some future, p forever	a path that can stay good
`AX p`, `EX p`	every / some next step satisfies p	one-step consequence
`[p AU q]`	every path keeps p until q, and q happens	constrained progress
`[p EU q]`	some path keeps p until q, and q happens	possible progress
`[p AW q]`	every path keeps p until q, or keeps p forever	"not before"
`[p EW q]`	some path keeps p until q, or keeps p forever	a path that can wait
`never p`	`AG not p`	forbidden behavior
`p ==> q`	`EF p` and `AG (p => q)`	non-vacuous implication

Until is written infix, with its quantifier attached: `[p AU q]`, never `A[p U q]`. The second form looks like an LTL formula placed inside a quantifier, but CTL has no path formulas of its own. `AU` is one operator, just as `AG` and `EF` are, and this is the form LeanFM uses in its JSON (`{"AU": [p, q]}`).

Atoms are events

A message name, like `Get`, holds at the point just after a `Get` was delivered. `terminal` holds when the protocol has finished. A finished run stutters in place forever, so every path is infinite and `AF` and `AG` have their textbook meaning. Atoms never mention hidden state: if a requirement needs a fact no event reveals, the fix is to add the event.

Scope: what an attached formula talks about

> **The attachment rule**
> A formula attached to protocol `P` (an entry `("P", φ)` in `ctl`) must hold at the moment any instance of `P` begins, over every continuation of that instance: its own events, everything it includes, and every repetition, until the instance is finished. For `Request`, finished means its reply. For `Work`, which continues itself, it means the end of the session.

This puts each formula where it is true. "Every request is answered" is `AF terminal` on `Request`, a two-event fact checked on a three-state model. On `Session` the same idea needs the longer `Get ==> AF (Page or NotFound)`. Local formulas also give short counterexamples.

Idioms

`AG EF terminal`	no trap: finishing is always still possible
`AF terminal`	always finishes (often *too strong*, see Chapter 8)
`[ not x AW y ]`	no `x` before `y`
`EF x`	`x` is reachable; the witness is a usage scenario
`never x`	`x` is forbidden; pair it with the positive `AG` it protects
`x ==> AF y`	every `x` is followed by `y`, and `x` can actually happen

LeanFM reads the Session's first formula aloud as: *on every path, (it is not the case that Get was just delivered) holds unless Welcome was just delivered (Welcome was just delivered need not happen)*. Its negation, which is what a counterexample must show, reads: *on some path, (it is not the case that Welcome was just delivered) holds until (Get was just delivered and (it is not the case that Welcome was just delivered)) (and (Get was just delivered and (it is not the case that Welcome was just delivered)) does happen)*.

6 Time

CTL stays untimed. Time enters through timestamps on events and `where` clauses on protocols.

A `where` clause is Lean: a `Bool` over the payloads of the event it guards and of earlier named events. Two patterns cover most timing requirements:

Bounded response

`p: Page from Server to Client cause g where p.time - g.time ≤ 2000;`. Together with `AF terminal` on `Request`, this gives bounded liveness: a reply always comes, and comes within two seconds.

Timeout as an event

`x: Expired from Server to Client where x.idle ≥ 1800000;`. A timeout is not a hidden timer. It is an event, enabled by the idle gap since the previous event on this thread.

Because the bound lives on the event that ends the interval, and the interval's start is named by a link, a duration is always unambiguous, even with many overlapping requests.

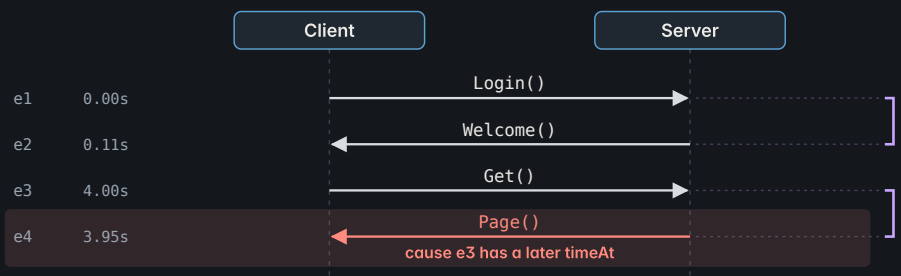


Figure 6.1 A reply stamped 50 ms *before* its request. The stream contradicts itself, whatever the protocol says.

> LeanFM `check_trace`: **inadmissible** — 4 events over 1 task(s); 0 concurrent pair(s) · e4: prev e3 has a later timeAt (4000 > 3950)

What is checked today

LeanFM's `check_trace` enforces that ids are unique, that links point to earlier events, that no link runs backward in time, that links are acyclic, and that each task is accepted by its compiled protocol. A `where` clause is type-checked when the protocol is written, and can be evaluated over decoded payloads; but when the model checker builds the futures tree, it treats it as a free choice: `valid l.user` may go either way at every login. And a clause sees payloads, not the envelope: the examples carry `time` and `idle` as fields until the event's own `timeAt` and `prev` are exposed to clauses.

7 Checking: from protocols to verdicts

A protocol compiles to a finite transition system. CTL is decided on it by fixed-point iteration, and the answer comes with a path.

Lean elaborates a protocol to a `Global` value, with every `include` already in place, and exports it. Compiling it turns each `continue` into an edge back to the start of its `rec` block, so recursion becomes a cycle. Equivalent states are then merged. `Work` becomes:

from	event	to
s0 ▸	`Client → Server : Get`	s1
s0 ▸	`Client → Server : Logout`	s2
s0 ▸	`Server → Client : Expired`	s3 ■
s1	`Server → Client : Page`	s0
s1	`Server → Client : NotFound`	s0
s2	`Server → Client : Bye`	s3 ■

▸ initial state · ■ terminal state. `continue Work` after `Page` or `NotFound` is the edge back to `s0`.

That table *is* the task model sent to LeanFM: states, an initial state, terminal states, and transitions labeled with message, sender and receiver. LeanFM builds its reachable nodes (a state plus the message that reached it, so that atoms like `get` can be evaluated) and computes each formula by least or greatest fixed point. A *universal* claim that fails comes back with a counterexample path. An *existential* claim that holds comes back with a witness. When the path loops, LeanFM says where it loops back.

Results for the session protocol

Production	Formula	LeanFM verdict	Evidence
Session	`[ not Get AW Welcome ]`	**proved**	all 10 nodes
Session	`[ not Get AU Welcome ]`	**disproved**	counterexample: Login · Denied ↻
Session	`EF Bye`	**proved**	witness: Login · Welcome · Logout · Bye
Session	`AG EF terminal`	**proved**	all 10 nodes
Session	`Get ==> AF (Page or NotFound)`	**proved**	witness: Login · Welcome · Get
Work	`AG EF terminal`	**proved**	all 7 nodes
Work	`AF terminal`	**disproved**	counterexample: Get · Page ↻
Request	`AF terminal`	**proved**	all 4 nodes

Production	Formula	LeanFM verdict	Evidence
FetchPage	`AF terminal`	**proved**	all 5 nodes
FetchPage	`EF Content`	**proved**	witness: Get · Fetch · Content

Two formulas are disproved, and both failures teach something. They are the subject of the next chapter.

8 Witnesses and counterexamples

Evidence is a stream, so evidence is an interaction diagram. A requirement is read through its witness and its counterexample.

A witness is a usage scenario

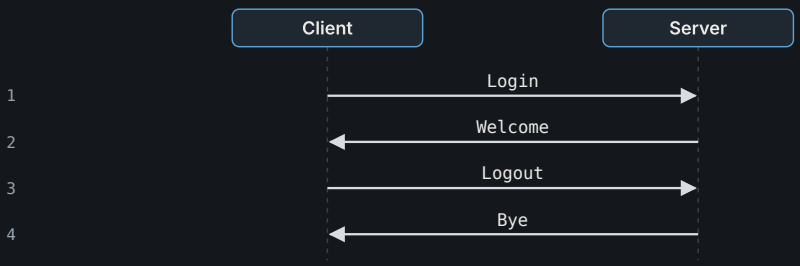


Figure 8.1 `EF Bye` on `Session` : the shortest path to a clean goodbye.

> LeanFM `check_ctl` : **proved** — it is possible that eventually Bye was just delivered

Every `EF` formula yields one of these. Collected together, they are the protocol's documentation by example, and they cannot go stale, because they are regenerated from the protocols.

A lasso is a forever

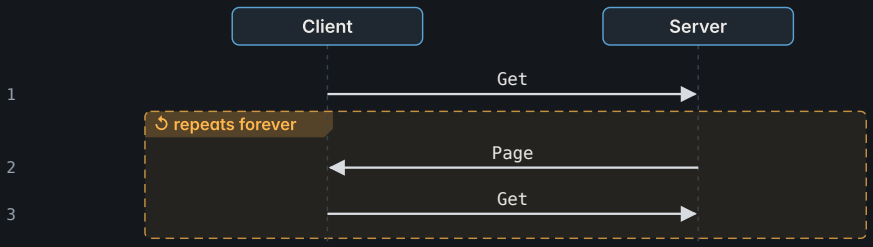


Figure 8.2 `AF terminal` on `Work` is disproved. After one `Get`, the client may answer every `Page` with another `Get`, forever.

> LeanFM `check_ctl` : **disproved** — on every path, eventually the task has ended

The checker is right, and the counterexample forces a decision: is endless browsing acceptable? For a web session it is, so the real requirement is the weaker `AG EF terminal` (finishing always stays possible), which is proved. Had the answer been no, the fix would have been in the protocol, for example a bounded number of requests.

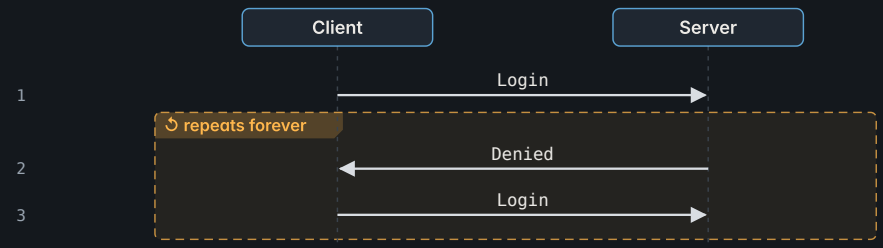


Figure 8.3 `[ not Get AU Welcome ]` is disproved by a client that is denied forever. Strong until demands that `Welcome` actually happens.

> LeanFM `check_ctl` : **disproved** — on every path, (it is not the case that Get was just delivered) holds until Welcome was just delivered (and Welcome was just delivered does happen)

The intent was only "no request before a welcome". That is weak until, `[ not Get AW Welcome ]`, which is proved. The difference between `AU` and `AW` is exactly the forever path in the figure.

9 Finding a bug

Change one line in the protocol. The formulas catch it before any code exists, and the trace check catches it after.

Suppose a server is written so that a denied login falls through into the work loop: one line changed, `continue Login` to `include Work Client Server`.

```
/-- One call changed: a denied login falls through into the work loop.
-/
protocol SessionBug (Client, Server) {
  rec Login {
    l: Login from Client to Server;
    choice {
      Welcome from Server to Client cause l where valid l.user;
      include Work Client Server;
    } or {
      Denied from Server to Client cause l where !valid l.user;
      include Work Client Server;
    }
  }
}
```

Before any code exists, the Session's first formula fails on the edited protocol:

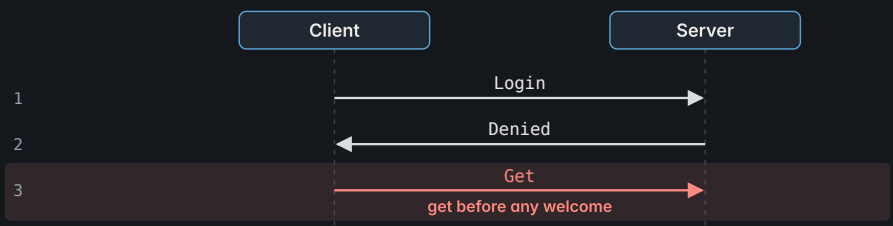


Figure 9.1 Counterexample to `[ not Get AW Welcome ]` on the edited protocol: a request is served after a denial.

> LeanFM `check_ctl`: **disproved** — on every path, (it is not the case that Get was just delivered) holds unless Welcome was just delivered (Welcome was just delivered need not happen)

After code exists, the same mistake shows up in a recorded stream. Checked against the *correct* protocol, the trace is rejected at the first event the protocol does not allow, and LeanFM reports what it expected instead:

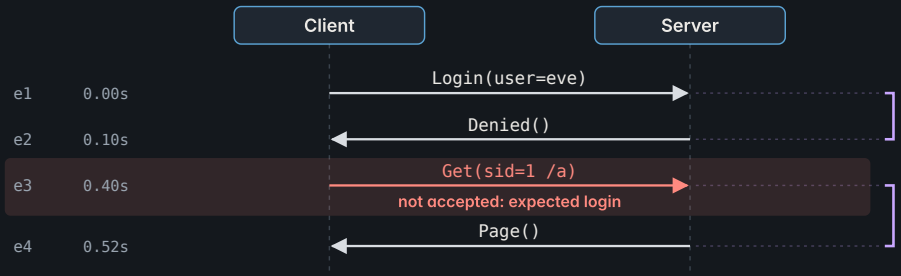


Figure 9.2 A stream from the buggy server, checked against the specification.

> LeanFM `check_trace` : **inadmissible** — 4 events over 1 task(s); 0 concurrent pair(s) · task Session: event e3 is not accepted by the task's transitions (expected: Login)

The same specification produced both results: a counterexample from the model before implementation, and a rejection from the stream after it. Note what did *not* catch it: the session checker. The edited protocol is still well formed, every choice still has one decider; it is just the wrong protocol. Well-formedness and requirements are different questions, and the formulas answer the second.

10 Past and future

The stream remembers; CTL anticipates.

Past: follow links

The history behind an event is the set of events reachable backward through cause and prev. It is a fixed fact, a single history, so no quantifier is needed. `where` clauses read it: `p.time - g.time ≤ 2000` reaches back to the request `p` answers; `p.sid = g.sid` says this reply carries the session its request named.

Future: quantify paths

The future after a prefix is a tree, because protocols branch. `A` and `E` range over its branches; `F`, `G`, `X`, `U` and `W` range along a branch. Possibility (`EF`) and necessity (`AF`, `AG`) stay distinct, and LTL cannot express that distinction.

Keeping the past as data means a requirement never has to restate history as hidden state. Two runs that look identical on the stream cannot be told apart by the specification. If the future must depend on something, that something must have been an event.

The same division gives runtime monitoring, and it needs no state. Nothing about the system is stored between events. Consuming an event moves the present forward, and at each present we make assertions: about the past, by following links in the prefix (which requests are still unanswered, how long a reply took), and about the future, by asking each formula again of what the protocol still allows. The position a prefix reaches in the compiled protocol is a place in the protocol, not a record of system state; it only says which continuations remain.

Safety (`AG`, `never`) fails on the first bad event, as in Figure 9.2. A possibility (`EF`) can be found impossible early, as soon as nothing that may follow leads to it. An obligation (`AF`) is asserted at the present that creates it, such as `AF (Page or NotFound)` right after a `Get`, and stays open while the run continues; the monitor reports what is still owed. Today `check_trace` does the acceptance part for a complete recorded stream, and the video's per-event assertions are `check_ctl` started at each prefix's position; running this on a live stream is the same computation applied event by event.

11 Working this way

Requirements are the public contract. Everything else is implementation detail and can be regenerated.

1. **Name the actors and messages that cross boundaries.** If no event reveals a fact, it is not a requirement yet.
2. **Write protocols.** Give every message a Lean structure. Use `rec` for every repetition and `include` to stack protocols. Put a `cause` on every reply, and name the events later clauses read. Let Lean's session checker settle who decides each choice before anything else.
3. **Attach the positive invariant first**, usually an `AG`. Then the forbidden behavior, as `never` or `[not x AW y]`. Then the possibilities, as `EF`.
4. **Check.** Read every witness as a scenario and every counterexample as a question. Fix either the protocol or the formula, whichever was wrong.
5. **Freeze what others depend on.** Once a client relies on a protocol, it is public. Persistent implementation choices (storage layout, byte encodings) go in a separate implementation specification.
6. **Implement in one pass** from the checked specifications, then check recorded streams against the same protocols.

This suits working with an LLM. Questions in a requirements conversation should either pin down public behavior or record a persistent implementation decision. The diagrams and verdicts are the review artifacts, and git is where they are accepted. When the specification is complete, the chat history can be thrown away: the book, the protocols and their checks are the durable statement of what to build.

A Notation

The protocol syntax comes from LeanFM's `LeanFM.Session` library. Its words are not reserved, so importing it takes no identifiers away from Lean code.

```
protocol   ::= 'protocol' Name '(' Role (',' Role)* ')' '{' stmt* '}'
stmt       ::= [id ':'] Msg 'from' Role 'to' Role ['cause' id] ['where' term] ';'
             | Event 'at' Role ';'                    -- an event at one role only
             | 'choice' 'at' Role '{' stmt* '}' ('or' '{' stmt* '}')*
             | 'par' '{' stmt* '}' ('and' '{' stmt* '}')*
             | 'join' n 'of' '{' stmt* '}' ('and' '{' stmt* '}')*
             | 'rec' Name '{' stmt* '}'
             | 'continue' Name ';'
             | 'include' term ';'                     -- another protocol, with roles

formula    ::= f ['=>' formula | '==>' formula]
f          ::= g ('or' g)*        g ::= u ('and' u)*
u          ::= 'not' u | 'never' u
             | ('AG'|'AF'|'EG'|'EF'|'AX'|'EX') u
             | '[' formula ('AU'|'EU'|'AW'|'EW') formula ']'
             | '(' formula ')' | 'true' | 'false' | 'terminal' | Msg
```

- `Msg` is the name of a Lean type, the protocol's vocabulary. A misspelt message is an unknown identifier.
- `where term` is a Lean `Bool` over the named events in scope, the guarded one included; it is type-checked when the protocol is written.
- `protocol P (A, B) {…}` defines `P : R → R → Global R` for any role type `R`, so a protocol can be included with different roles.
- A `cause` must name an event its sender has sent or received: a role decides only from its own history. Lean checks this, together with the choice rule (every branch starts with something its decider does) and projection (every role can tell which branch it is in, unless its part of the session has not started yet).
- `continue` ends its branch, so repetition is always in tail position.
- `par` and `join` belong to the library. The companion compiler used for this book does not yet expand them into task models.

B JSON forms

These are the exact shapes passed to LeanFM's MCP tools. They are also what an implementation's logs should contain.

Event (`check_trace`)

```
{ "id": "b2", "prevIds": ["b1"], "causeIds": ["b1"], "task": "bob", "src": "Server",
"dst": "Client(bob)", "message": "Denied", "timeAt": 150 }
```

Formula (`check_ctl`)

`[ not Get AW Welcome ]` becomes

```
{ "AW": [{ "not": { "after": "Get" } }, { "after": "Welcome" }] }
```

`Get ==> AF (Page or NotFound)` becomes

```
{ "stronglyImplies": [{ "after": "Get" }, { "AF": { "or": [{ "after": "Page" }, {
"after": "NotFound" }] } }] }
```

Task model: the compiled `Work` protocol

```
{"id": "Work", "initial": "s0",
 "states": ["s0","s1","s2","s3"], "terminals": ["s3"],
 "transitions": [
  {"src":"s0","dst":"s1","message":"Get","from":"Client","to":"Server"},
  {"src":"s0","dst":"s2","message":"Logout","from":"Client","to":"Server"},
  {"src":"s0","dst":"s3","message":"Expired","from":"Server","to":"Client"},
  {"src":"s1","dst":"s0","message":"Page","from":"Server","to":"Client"},
  {"src":"s1","dst":"s0","message":"NotFound","from":"Server","to":"Client"},
  {"src":"s2","dst":"s3","message":"Bye","from":"Server","to":"Client"}]}
```

C Reproducing this book

```
~/code/leanFM     LeanFM (git: origin/main), with LeanFM.Session
~/code/seLean     this book
  Examples/Session.lean  the running example, in session-type syntax
  Examples/Proxy.lean    nested brackets
  Examples/Export.lean   global types, checker issues, local types -> JSON
  examples/traces.mjs    the event streams
  tools/lean.mjs         Lean export -> productions
  tools/lfm.mjs          CTL parser, task-model compiler, frame matcher
  tools/seq.mjs          interaction-diagram renderer
  Gen.lean               exports the examples; runs check_ctl, check_trace
  build.mjs              checks, figures, PDF

./build.sh      # lake build gen, then node build.mjs
```

Lean is the only source of the protocols: `lake exe gen --examples` elaborates them, runs the session checker, and exports what the rest of the build uses. `Gen.lean` calls `LeanFM.Logic.run`, the function the LeanFM MCP server dispatches its logic tools to, so no server needs to be running. The build asserts every verdict that the text relies on. Changing a production so that a claim in the book becomes false makes the build fail.